

Project – Phase 2

1. Grammar (CFG)

Overall Structure:

program $\rightarrow$ START stmt_list END

START $\rightarrow$ shuru

END $\rightarrow$ khatam

Expressions:

expression $\rightarrow$ T_INT

expression $\rightarrow$ T_FLOAT

expression $\rightarrow$ T_ID

Conditions:

condition $\rightarrow$ expression T_{OP_EQ} expression

condition $\rightarrow$ expression T_{OP_LT} expression

condition $\rightarrow$ expression T_{OP_GT} expression

Statement list:

stmt_list $\rightarrow$ stmt stmt_list $\mid \epsilon$

Statements:

stmt -> T_KEYWORD T_ID T_SEMICOLON | T_ID T_OP_ASSIGN *expression*
T_SEMICOLON

stmt -> T_KEYWORD *condition* T_PUNC_BLOCK_START *stmt_list*
T_PUNC_BLOCK_END

stmt -> T_KEYWORD *condition* T_PUNC_BLOCK_START *stmt_list*
T_PUNC_BLOCK_END

2. FIRST & FOLLOW:

Non-terminal: stmt

FIRST(stmt):

T_KEYWORD, T_ID

FOLLOW(stmt):

T_KEYWORD, T_ID, T_PUNC_BLOCK_END, \$

Non-terminal: condition

FIRST(condition):

T_INT, T_FLOAT, T_ID

FOLLOW(condition):

)

3. PARSE TREE EXAMPLE (VALID):

shuru

kaam x;

x <- 10;

agar (x < 20)

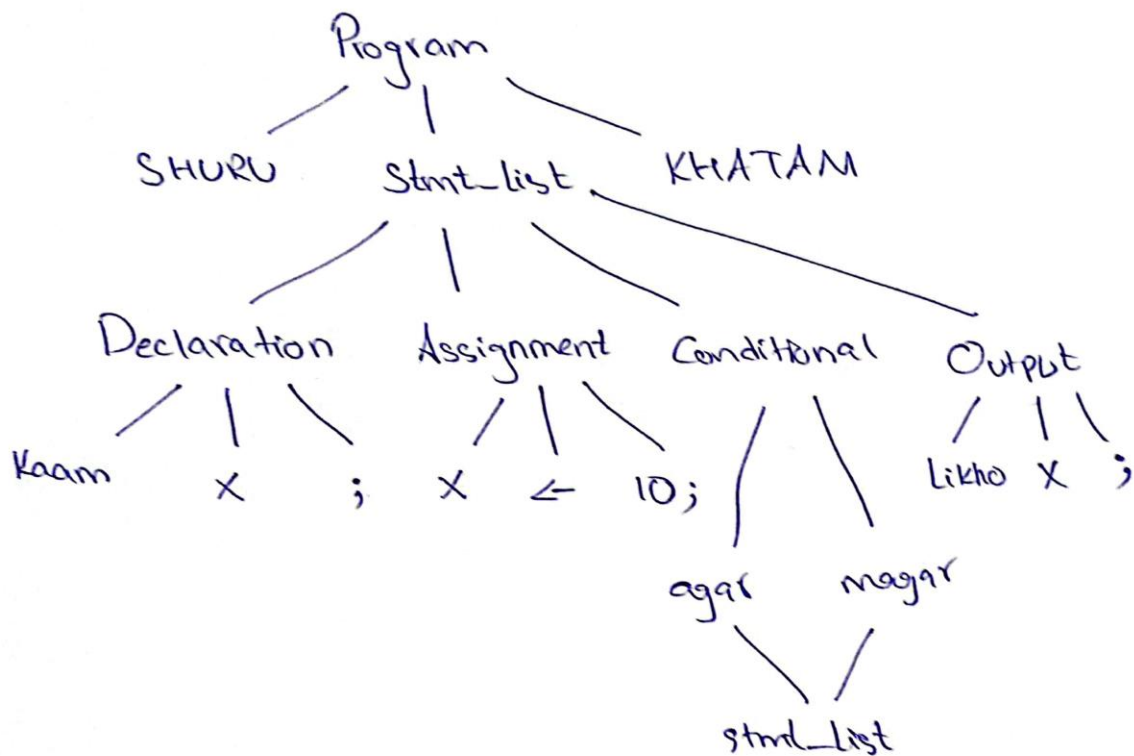
@{

likho x;

}@

khatam

Parse Tree of Sample Chosen :



4. Explanation of Phase 1 Tokens:

Token Name	Meaning / Lexeme
T_INT	Integer literals (e.g., 10)
T_FLOAT	Floating-point numbers (e.g., 3.14)
T_EXP	Exponential numbers (e.g., 1e10)
T_ID	Identifiers (variable names)
T_KEYWORD	Keywords (agar, magari, shuru, etc.)
T_OP_ASSIGN	Assignment operator <=
T_OP_ADD	Addition +
T_OP_SUB	Subtraction -
T_OP_MUL	Multiplication *
T_OP_DIV	Division /
T_OP_EQ	Equality ==
T_OP_LT	Less-than <
T_OP_GT	Greater-than >
T_PUNC_SCOPE	Scope punctuation ::
T_PUNC_BLOCK_START	Block start @{
T_PUNC_BLOCK_END	Block end }@
T_SEMICOLON	Statement terminator ;
T_COMMA	Comma ,
T_LPAREN	Left parenthesis (
T_RPAREN	Right parenthesis)
T_STRING	String literal "..."
T_CHAR	Character literal 'a'

Lexer (**scanner.l**) generates these tokens and writes to **tokens.txt**.

Parser (**parser.y**) reads these tokens via **yylex()** function.

All grammar rules are based on these already defined tokens.

Errors detected by parser (**syntax errors**) complement lexer errors.

5. Error Handling

- **Lexer Errors (Phase 1):** malformed numbers, invalid identifiers, unclosed strings/characters, unrecognized symbols.

- **Parser Errors (Phase 2):**

Missing program start (shuru) or end (khatam) → prints Line X: Expected 'shuru'/'khatam'.

Invalid statement structure -> prints Line X: Unexpected keyword in statement.

Optional else mismatch -> prints Line X: Expected 'magar' for else block.

Both lexer and parser output errors in line number + message format.